Reporte

EQUIPO 13	
Nombre: Edgar Jesus Del Angel Martinez Israel Alejandro Pantoja Esquivel Andrea Fernanda Rodríguez Niño	**Matrícula:** 2160745 2155971 2160663
Nombre del curso: Componentes de Lógica de Negocio	**Nombre del profesor:** Miguel Ángel Gómez Marroquín
Módulo NetAcad: 4	**Actividad:** Laboratorio guiado. Programa para administrar pedidos de una tienda ficticia TechStore.
Fecha: 28/10/2026	

Bibliografía (formato APA)

- Beck, K. (2002). Test-driven development: By example. Addison-Wesley Professional.
- Evans, E. (2003). Domain-driven design: Tackling complexity in the heart of software. Addison-Wesley.
- Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley.
- Millett, S., & Tune, N. (2015). Patterns, principles, and practices of domain-driven design. Wrox.
- Vernon, V. (2013). Implementing domain-driven design. Addison-Wesley Professional.

Objetivo:

El estudiante implementará los componentes de lógica de negocio diseñados en U3: entidades DDD, reglas con Specification Pattern, máquinas de estado, Domain Events y APIs REST.

Procedimiento:

Procedimiento y desarrollo

El proyecto se organizó separando las reglas del negocio de la API. Primero se trabajó el modelo del pedido y sus validaciones; después se definieron las Specifications para los descuentos y las operaciones permitidas según el estado. Finalmente, se conectaron esas piezas con el servicio, FastAPI y los archivos CSV usados para cargar los datos de prueba.

La información inicial se encuentra en la carpeta datos/. El archivo cargar_csv.py convierte los registros de clientes, productos y pedidos en objetos que utiliza el dominio. La aplicación mantiene los cambios en memoria, por lo que al reiniciarla vuelve a cargar los datos originales.

1. Entidades y objetos de valor

El dominio está implementado en app/domain.py. Money representa importes en MXN y utiliza Decimal para evitar los errores de precisión que pueden aparecer al trabajar con float. También valida que el importe sea finito y no negativo, y que las operaciones monetarias se realicen entre monedas compatibles.

OrderItem representa un artículo de un pedido mediante SKU, cantidad y precio unitario. Customer contiene el identificador del cliente y las condiciones de cliente VIP y activo. Order es la entidad que concentra el pedido, calcula el total y protege las reglas que deben cumplirse durante su ciclo de vida.

Componente	Uso en PetMarket
Money	Representa cantidades monetarias con Decimal; no admite valores negativos ni importes no finitos.
Customer	Identifica al cliente y conserva si es VIP y si está activo.
OrderItem	Representa un SKU y su cantidad dentro del pedido; calcula el subtotal.
Order	Aggregate Root del pedido; administra artículos, total, estado y eventos.
DomainException	Base de las excepciones usadas cuando se incumple una regla del negocio.

Excepciones de dominio

Excepción	Cuándo se utiliza	Respuesta de API
InvalidOrderData	Datos inválidos o precondición no cumplida, como pedido vacío, cliente inactivo o envío sin guía.	422
InvalidOrderTransition	La operación no corresponde al estado actual, por ejemplo confirmar dos veces o cancelar un pedido enviado.	409
OrderItemLimitExceeded	La suma de unidades excedería el máximo permitido por pedido.	422

2. Specification Pattern

Las reglas de descuento están en app/specifications.py. Cada Specification evalúa una condición del pedido sin modificarlo. Las condiciones pueden combinarse con AND (&),

OR (|) y NOT (~), lo que permite expresar las reglas de manera separada y probarlas individualmente.

Regla	Condiciones	Descuento
VIP	Cliente VIP, total mayor a $500 MXN, mínimo 4 unidades y sin promoción activa. Todas las condiciones deben cumplirse.	10 %
Importe alto	Total mayor a $5,000 MXN y sin promoción activa. No depende de que el cliente sea VIP.	5 %
Ninguna regla	Si no se cumple alguna de las dos condiciones anteriores.	0 %

La regla VIP se revisa primero. Así, si un pedido cumple los requisitos de ambas reglas, recibe el 10 %. El descuento y el total con descuento se calculan por separado; la evaluación de las Specifications no cambia el total original ni registra eventos.

3. Máquina de estados del pedido

Los pedidos se cargan inicialmente en DRAFT. Para cambiar de estado, el método correspondiente valida primero las condiciones, después actualiza el estado y al final registra el evento. Si la validación falla, el pedido conserva su estado y no se agrega un evento nuevo.

Transición	Operación	Condición principal	Evento
DRAFT → CONFIRMED	confirm()	Pedido con artículos y cliente activo.	OrderConfirmed
CONFIRMED → SHIPPED	ship(tracking_number)	Se proporciona una guía válida.	OrderShipped
CONFIRMED → CANCELLED	cancel(reason)	Se indica un motivo de cancelación.	OrderCancelled
SHIPPED → DELIVERED	deliver()	El pedido se encuentra enviado.	OrderDelivered
SHIPPED → RETURNED	return_order()	El pedido se encuentra enviado.	OrderReturned

DELIVERED, RETURNED y CANCELLED son estados finales en el flujo planteado. Las operaciones que no corresponden al estado actual son rechazadas, en lugar de cambiar el pedido de manera parcial.

4. Domain Events

Los eventos representan hechos que ya ocurrieron dentro del negocio. Se registran al completar una transición válida, por ejemplo cuando un pedido queda confirmado o sale con una guía. No se generan eventos para intentos rechazados, ya que el estado del pedido no cambió.

Evento	Qué representa
OrderConfirmed	El pedido fue confirmado.
OrderShipped	El pedido fue enviado con número de guía.
OrderCancelled	El pedido fue cancelado antes de enviarse.
OrderDelivered	El pedido fue entregado.
OrderReturned	El pedido enviado fue devuelto.

5. API REST con FastAPI

La API se encuentra en app/api.py y utiliza app/service.py para coordinar las operaciones del pedido. El servicio recupera la orden en memoria y llama al método del dominio; la API transforma el resultado en JSON y presenta los errores de negocio con códigos HTTP.

FastAPI genera la documentación interactiva en Swagger UI. Al iniciar el servidor con Uvicorn, la interfaz queda disponible en /docs, desde donde se pueden consultar pedidos y probar las operaciones disponibles.

Para ejecutar la aplicación desde la carpeta del proyecto:

```
.venv\\Scripts\\python.exe -m uvicorn app.api:app --host 127.0.0.1 --port 8000
```

Las respuestas de error mantienen una estructura común que incluye el tipo de error, el mensaje y el identificador del pedido. Esto permite distinguir, por ejemplo, un pedido inexistente (404), una transición no permitida (409) y datos inválidos (422).

Ejemplos de funcionamiento

1. Pedido O001: cálculo del descuento

O001 corresponde a un cliente VIP activo, no tiene promoción y contiene cuatro unidades. El total original es de $2,100.00 MXN. Como el cliente es VIP, el importe supera $500, se alcanza el mínimo de cuatro unidades y no hay promoción activa, se cumple la regla del 10 %.

Concepto	Cálculo	Importe
Total original	2 × $600.00 + 2 × $450.00	$2,100.00 MXN
Descuento	$2,100.00 × 10 %	$210.00 MXN
Total con descuento	$2,100.00 − $210.00	$1,890.00 MXN

La evaluación del descuento no cambia el total original del pedido. Cuando se confirma la orden, el estado pasa a CONFIRMED y se registra OrderConfirmed.

2. Pedido O014: intento de cancelar después del envío

En este caso se confirma y se envía el pedido antes de intentar cancelarlo. Una vez que se encuentra en SHIPPED, cancel() no está permitida. La operación se rechaza con InvalidOrderTransition, el pedido conserva el estado SHIPPED y no se registra OrderCancelled. El resultado corresponde a la regla que limita la cancelación al estado CONFIRMED.

Pruebas y validaciones

1. Resultados de ejecución

Se ejecutó la suite automatizada con pytest desde el entorno del proyecto. La ejecución registrada terminó con 232 pruebas aprobadas. También se ejecutó el programa de escenarios, que procesa los casos E01 a E15 de manera aislada y reporta que los 15 cumplen su objetivo.

Comprobación	Resultado
Carga de archivos CSV	16 pedidos cargados desde los datos del proyecto.
Escenarios de negocio E01–E15	15 de 15 escenarios cumplen el resultado esperado.
Pruebas automatizadas	232 passed.

2. Qué cubren las pruebas

Las pruebas revisan la validación de importes y cantidades, las reglas de descuento y sus límites, las transiciones entre estados, el registro de eventos y las respuestas de la API. También verifican que una operación rechazada no modifique el pedido ni agregue un evento que represente una acción que no ocurrió.

En particular, se comprueba el límite de 20 unidades por pedido: se suman las cantidades de los artículos y no solamente el número de renglones. También se prueban los casos de pedido vacío, cliente inactivo, envío sin guía, confirmación repetida y cancelación después del envío.

Bitácora de escenarios E01–E15

La bitácora se genera con ejecutar_escenarios.py --bitacora a partir de datos/escenarios.csv. Cada escenario vuelve a cargar los archivos de datos, de modo que los cambios de un caso no afectan al siguiente. La tabla resume el estado inicial, la operación, el resultado observado y la razón del comportamiento.

Caso	Estado inicial	Operación	Resultado real	Explicación
E01	O001 · DRAFT; VIP activo; 4 u.; $2,100 MXN	Evaluar descuento	10 %; desc. $210; total $1,890. Sin eventos.	Cumple VIP, importe > $500, al menos 4 unidades y no hay promoción.
E02	O002 · DRAFT; 2 u.; $1,050 MXN; no VIP	Evaluar descuento	0 %; total $1,050. Sin eventos.	No cumple regla VIP y el total no supera $5,000.
E03	O003 · DRAFT; cliente inactivo; 4 u.	Confirmar	Rechazo 422; permanece DRAFT; sin OrderConfirmed.	La confirmación exige cliente activo.
E04	O005 · DRAFT; VIP; promoción activa; 4 u.	Evaluar descuento	0 %; sin eventos.	La promoción activa bloquea las reglas de descuento.
E05	O006 · DRAFT; 1 u.; $18,000 MXN	Evaluar descuento	5 %; descuento $900; total $17,100. Sin eventos.	El total supera $5,000 y no hay promoción; se aplica la regla alternativa.
E06	O007 · DRAFT; pedido vacío	Confirmar	Rechazo 422; permanece DRAFT; sin evento.	No se puede confirmar una orden sin artículos.
E07	O008 · DRAFT; 20 u.	Confirmar	Éxito; CONFIRMED; OrderConfirmed.	20 unidades están dentro del máximo permitido.
E08	O009 · DRAFT; 19 u.	Agregar SKU003 × 2	Rechazo 422; conserva 19 u.; sin evento.	La suma sería 21 unidades, por encima del límite de 20.
E09	O010 · DRAFT; 2 u.; $500 MXN	Evaluar descuento	0 %; sin eventos.	El importe debe ser mayor a $500; la comparación es estricta.

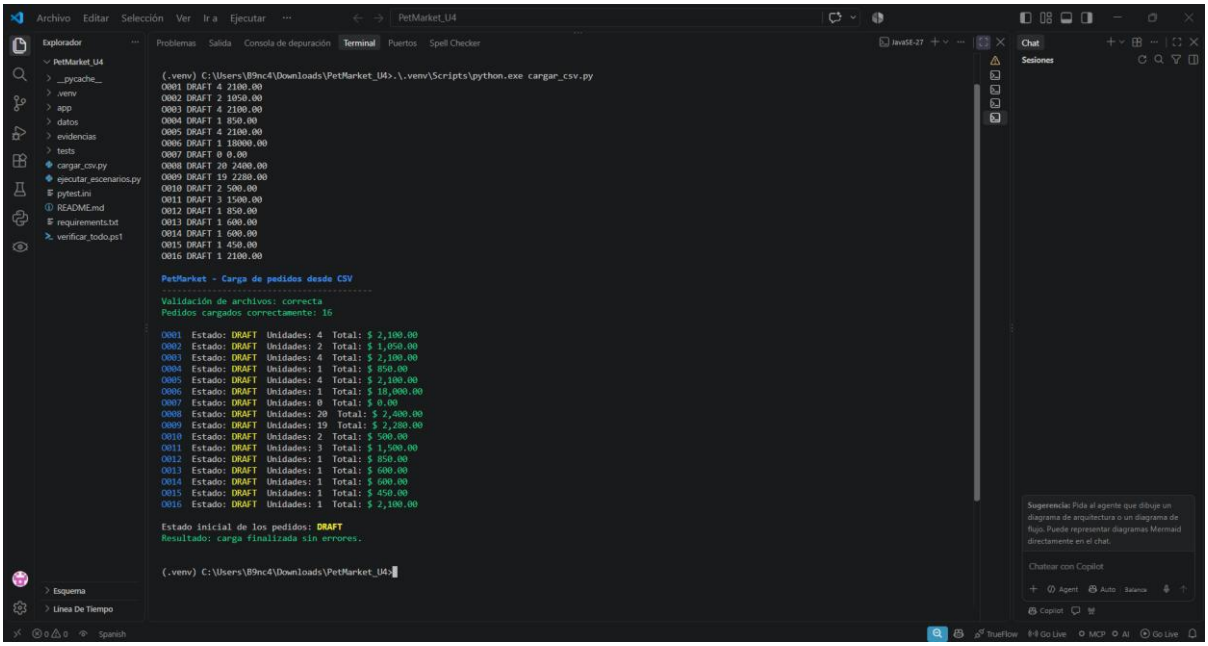
E10	O011 · DRAFT; VIP; 3 u.; $1,500 MXN	Evaluar descuento	0 %; sin eventos.	No alcanza las 4 unidades de la regla VIP ni supera $5,000.
E11	O012 · DRAFT; 1 u.	Confirmar y enviar sin guía	Confirmación válida; envío rechazado 422; CONFIRMED; solo OrderConfirmed.	El envío requiere número de guía; el pedido queda confirmado.
E12	O013 · DRAFT; 1 u.	Confirmar dos veces	Primera confirmación válida; segunda rechazada 409; CONFIRMED; un OrderConfirmed.	La confirmación solo se permite desde DRAFT.
E13	O014 · DRAFT; 1 u.	Confirmar, enviar y cancelar	Cancelación rechazada 409; SHIPPED; OrderConfirmed y OrderShipped.	No se puede cancelar un pedido que ya fue enviado.
E14	O015 · DRAFT; 1 u.	Confirmar, enviar y devolver	RETURNED; OrderConfirmed, OrderShipped y OrderReturned.	La devolución se permite desde SHIPPED.
E15	O016 · DRAFT; 1 u.	Confirmar, enviar y entregar	DELIVERED; OrderConfirmed, OrderShipped y OrderDelivered.	Recorre el flujo normal hasta la entrega.

Evidencias

Las imágenes siguientes corresponden a las capturas que se incluyen en la carpeta evidencias del proyecto. Se usan para mostrar la carga de datos, la ejecución de los escenarios y las pruebas, además de las respuestas de la API.

1. Evidencia 1 · Carga de datos

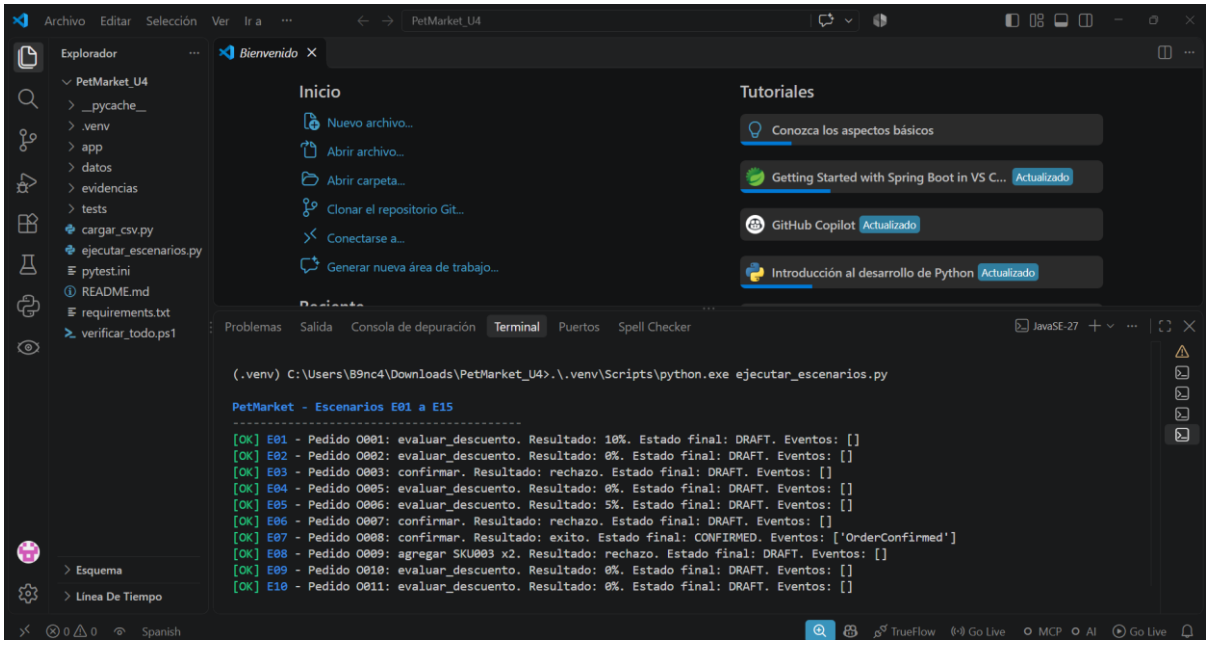
La evidencia corresponde a la carga inicial de la información del proyecto. El script lee los archivos CSV de clientes, productos y pedidos ubicados en la carpeta datos/, y utiliza sus registros para construir los objetos que necesita la aplicación. Esta preparación permite que los escenarios trabajen con información definida en los archivos del proyecto, en lugar de depender de datos escritos manualmente durante cada prueba. La carga también sirve como verificación inicial de que los archivos de entrada están disponibles y pueden ser procesados por el programa.



01 carga csv.png

2. Evidencia 2 · Inicio de escenarios

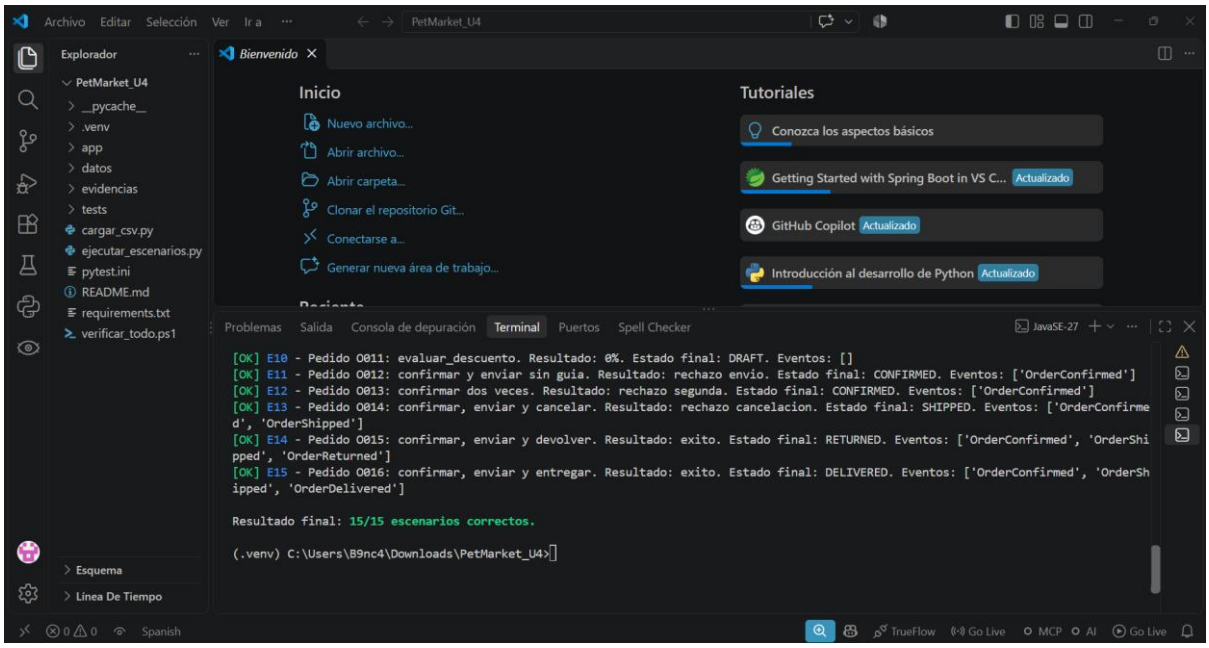
En esta captura se observa el inicio de la ejecución de los escenarios definidos para PetMarket. El programa toma los casos de datos/escenarios.csv y prepara cada uno para comprobar una regla concreta del sistema. Los escenarios incluyen condiciones normales, como confirmar un pedido que cumple los requisitos, y situaciones límite o inválidas, como superar el número máximo de unidades o intentar una operación desde un estado que no corresponde. La ejecución organizada por casos facilita relacionar cada resultado con la condición que se está evaluando.



02 escenarios inicio.png

3. Evidencia 3 · Resultado de escenarios

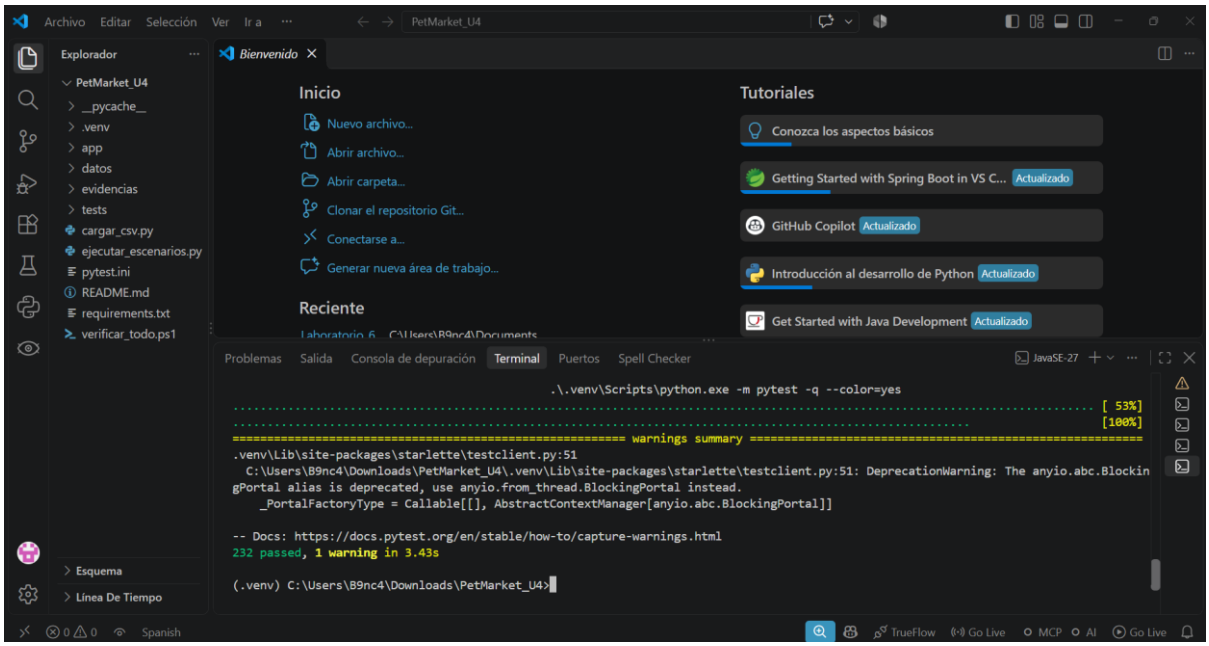
Esta captura muestra el cierre de la ejecución de los escenarios E01 a E15. El resultado permite revisar si el comportamiento obtenido coincide con lo esperado para cada caso. En conjunto, los escenarios cubren las reglas de descuento, el límite de unidades, las validaciones necesarias para confirmar o enviar un pedido y las transiciones hacia estados como SHIPPED, DELIVERED o RETURNED. La salida final del programa sirve como comprobación general del recorrido de los casos incluidos en el archivo de escenarios.



03 escenarios final.png

4. Evidencia 4 · Pruebas automatizadas

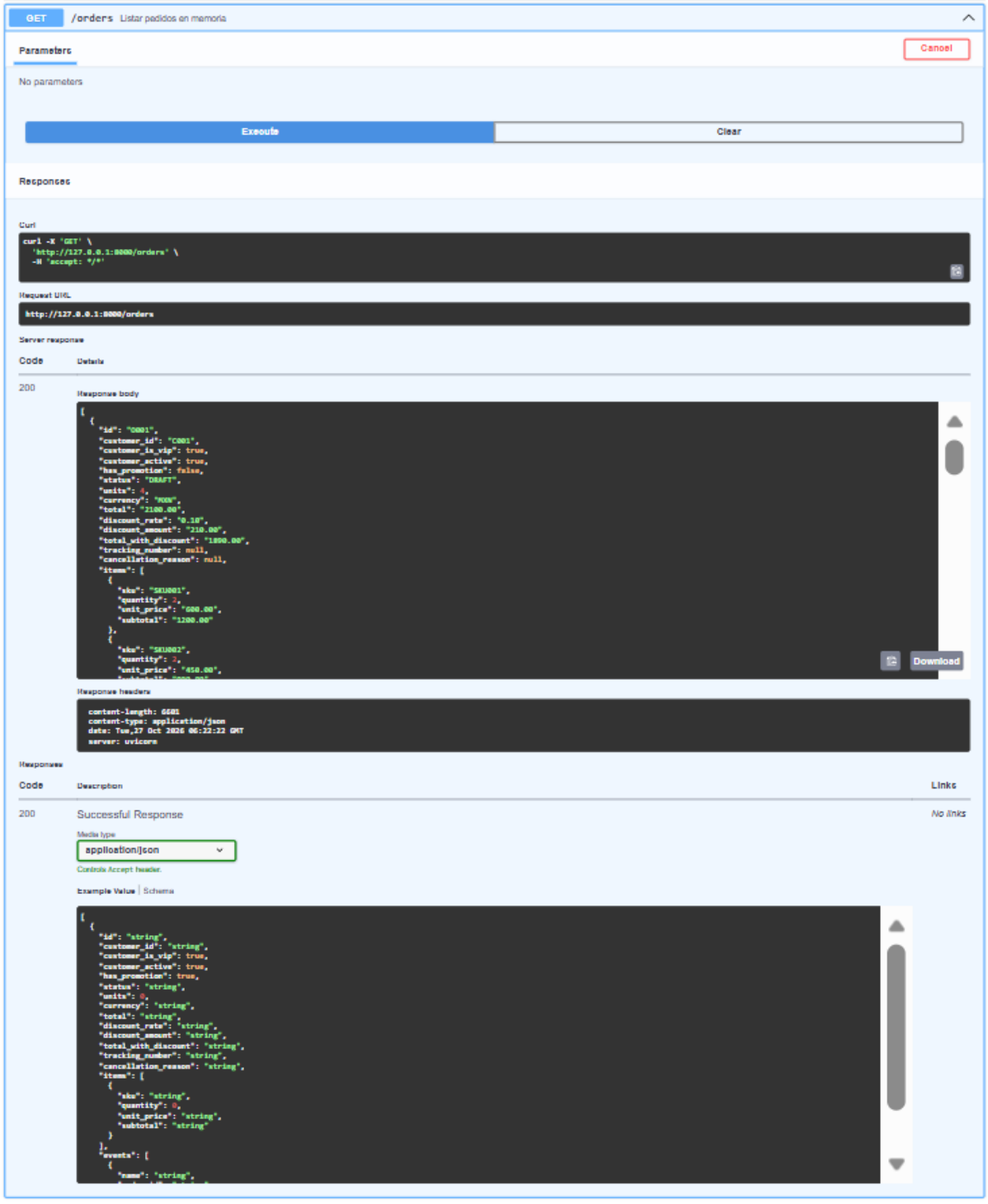
La imagen muestra la ejecución de las pruebas automatizadas con pytest. Estas pruebas verifican componentes del dominio y del servicio, además de revisar las respuestas de la API ante operaciones válidas e inválidas. La suite incluye comprobaciones de cálculos monetarios, cantidades permitidas, reglas de descuento, transiciones de estado y eventos de dominio. El resultado que aparece en la terminal permite identificar si las pruebas finalizaron correctamente o si alguna validación requiere revisión; esta evidencia documenta la ejecución global de las pruebas del proyecto.



04 pytest completo.png

5. Evidencia 5 · Consulta en API

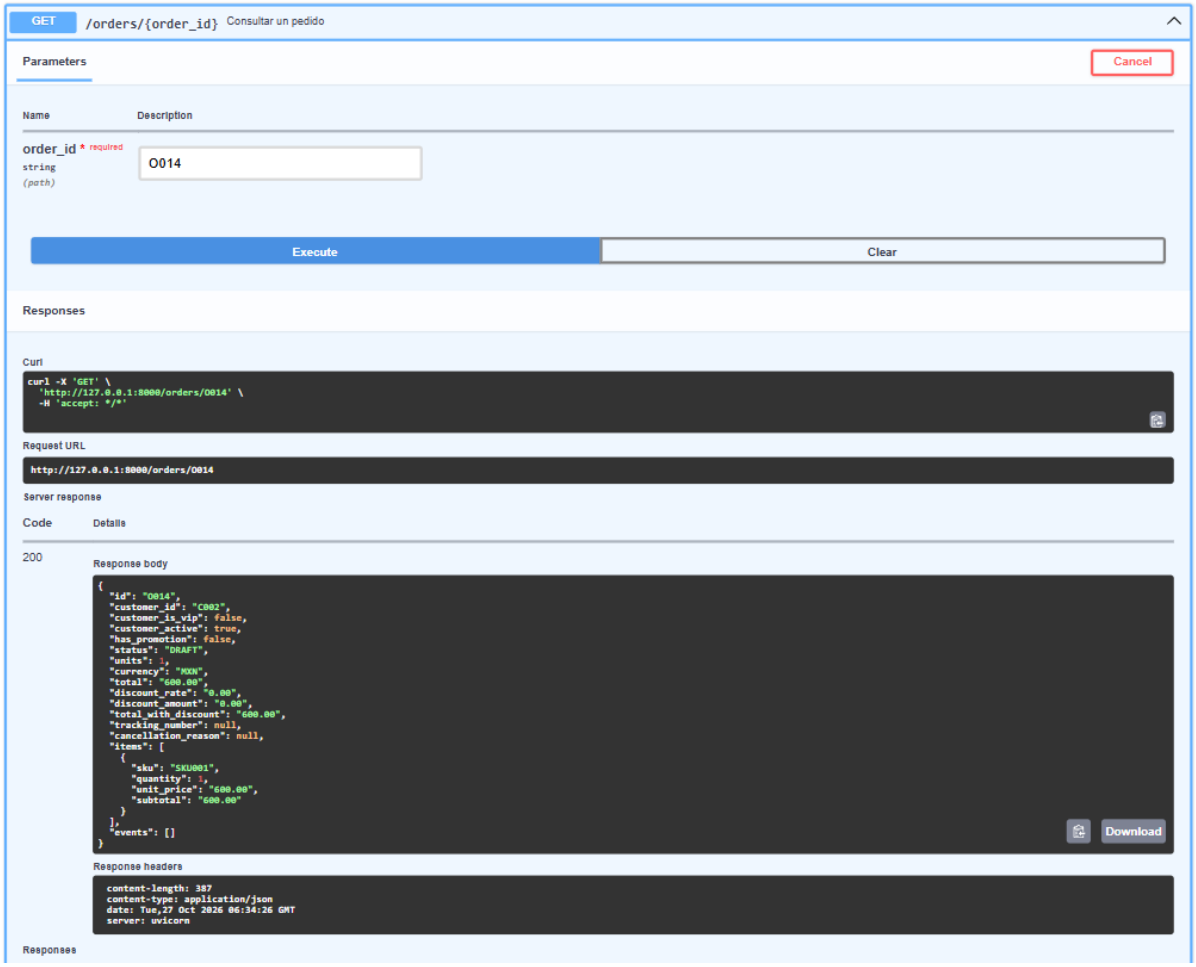
La captura presenta una consulta realizada desde Swagger UI, la documentación interactiva que FastAPI genera para la aplicación. Desde esta interfaz se puede seleccionar una ruta, proporcionar los parámetros requeridos y enviar la petición al servidor sin utilizar una herramienta externa. La consulta permite revisar cómo la API devuelve la información de un pedido en formato JSON y comprobar que la capa de presentación expone los datos administrados por el servicio y el dominio. Esta evidencia muestra el acceso de lectura a la información del sistema mediante una petición HTTP.



05 api consulta.png

6. Evidencia 6 · Confirmación de pedido

En esta evidencia se observa la petición utilizada para confirmar un pedido a través de la API. La operación llega al servicio, que solicita al dominio validar las condiciones de confirmación, entre ellas que el pedido contenga artículos y que el cliente cumpla las condiciones necesarias. Cuando la operación es válida, el pedido cambia de DRAFT a CONFIRMED y se registra el evento OrderConfirmed. La respuesta de la API permite comprobar que la operación fue procesada y que el cambio de estado se refleja en los datos devueltos.

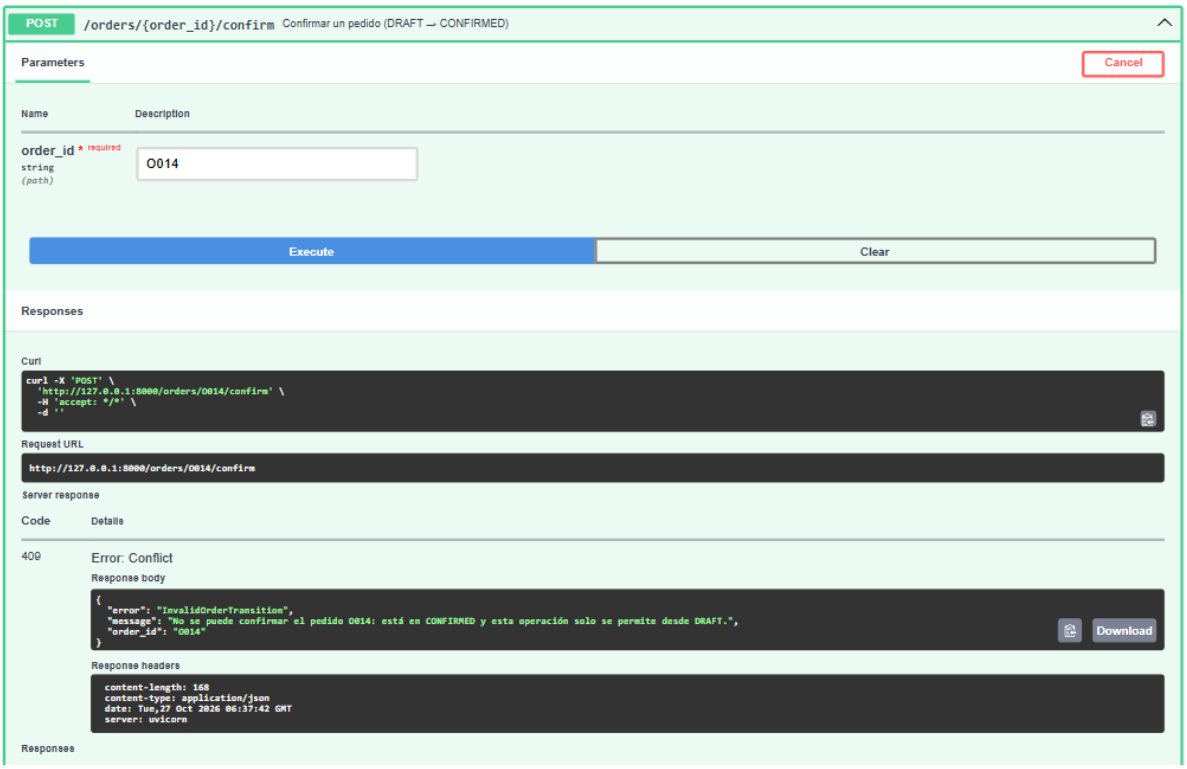


06 api confirmacion.png

7. Evidencia 7 · Respuesta de rechazo

Esta captura muestra una respuesta de error generada por la API ante una operación que no puede realizarse en el estado actual del pedido o que incumple una regla del negocio. En lugar de modificar la orden de forma incorrecta, el sistema devuelve un código HTTP de conflicto o de datos no válidos, acompañado de información que describe el motivo del rechazo. Este comportamiento ayuda a distinguir los errores de negocio de una

ejecución exitosa y mantiene la consistencia del pedido: si la operación no se completa, no se registra el evento asociado a esa transición.



07 api rechazo.png

Reflexión sobre el desarrollo

Durante el desarrollo fue importante separar las validaciones del dominio de las respuestas de la API. De esta manera, las reglas se aplican de la misma forma cuando se ejecutan desde las pruebas, el programa de escenarios o una petición HTTP. También se comprobó que las cantidades deben tratarse como unidades acumuladas y que las transiciones requieren validar el estado antes de registrar un evento.

Las pruebas ayudaron a revisar tanto los caminos correctos como las operaciones que deben rechazarse. En estos últimos casos, conservar el estado y no generar eventos nuevos evita que el sistema registre una acción que realmente no se completó.

Preguntas y respuestas

¿Por qué se utiliza Decimal en lugar de float para los precios?

Decimal permite representar y operar con importes decimales de manera adecuada para cálculos monetarios. float utiliza representación binaria y puede introducir pequeñas diferencias que afectan comparaciones o redondeos.

¿Cuál es la diferencia entre contar renglones y sumar unidades?

Un renglón representa un SKU dentro del pedido; las unidades son la suma de sus cantidades. El límite de PetMarket se aplica a las unidades totales, por lo que un renglón con cantidad 20 cuenta como 20 unidades.

¿Qué ventaja tiene usar Specifications?

Cada regla tiene una función clara y puede probarse por separado. Además, las condiciones se combinan para expresar los descuentos sin repetir la lógica en diferentes partes del sistema.

¿Por qué los eventos se registran después de una transición válida?

Un evento representa algo que ya sucedió. Si se registrara antes de validar la operación, podría quedar un evento de envío o cancelación aunque el pedido no hubiera cambiado de estado.

¿Qué se puede reutilizar si cambia la interfaz?

El dominio, las Specifications, el servicio, la carga de datos y las pruebas pueden mantenerse. La API es la parte que se adapta si se reemplaza FastAPI por otra interfaz.

Repositorio de GitHub

El código fuente, los archivos de datos, las pruebas y los recursos del proyecto PetMarket se encuentran disponibles en el siguiente repositorio:

Enlace al repositorio: https://github.com/EdgarDelAng/TechStore-U4-CLNEG-Equipo-13.git

El repositorio permite consultar la organización de los archivos y revisar el código utilizado para implementar las reglas de negocio, los escenarios y la API.